PROMPT:

2a)***This is the pseudocode we wish to use for A*** INPUT: Start Node s, Weight function w, heuristic h, successor generation function Expand, and predicate Goal OUTPUT: Cost optimal path from s to t (element of T), or empty if no such path exists closed <- empty Open <- s f(s) <- h(s) while (open is not empty) remove u from open with minimum f(u) insert u into closed if (Goal(u)) return Path(u) else Succ(u) <- expand(u) for each v in succ(u) Improve(u,v) return empty 2b)***This is the pseudocode for Improve, a void function*** INPUT: nodes u and v, v successor of u Side effects: update parent of v, f(v), Open, and Closed if v in Open if g(u) + w(u,v) < g(v) parent(v) <- u f(v) <- g(u) + w(u,v) + h(v) else if v in Closed if (g(u) + w(u,v) < g(v)) parent(v) <- u f(v) <- g(u) + w(u,v) + h(v) Remove v from Closed Insert v into Open with f(v) else parent(v) <- u Initialize f(v) <- g(u) + w(u,v) + h(v) Insert v into Open with f(v) 3)We are to implement the following Heuristics: 3a)Heuristic 1 is defined as the sum of the distances of the tiles to the goal, also known as the Manhattan distance heuristic) 3b)Heuristic 2 is defined as h2(s) we replace the tiles numbered 1 to 8 by a Red tile and the tiles numbered 9 to 15 by a Green tile. This maps each state of the puzzle to a state with 8 Red tiles, 7 Green tiles and an empty slot. The state space of this simplified board has size 16 × (15 C 8) = 102,960 which is significantly smaller than 16!/2 Let G be state space graph of the set of boards with Red and Green tiles. Let g' be the state in G to which the goal state g is mapped. For a state s in the original state, let s' be its mapped state in G. h2(s) = the distance between s' and g' in G. h2(s) is computed as follows: IDS is performed from g' to each state s' in G and is stored in a table. h2(s) is read off from the table entry for s' which is the state to which s is mapped under the red, green color mapping. There is an overhead associated with computing h2(s) but this is a one-time cost. Our existing code already has implemented IDS. We wish to reuse this algorithm instead of implementing a new one. 3c) Heuristic 3 is defined as h3(s) h3(s) is calculated by max{h1(s), h2(s)} The input output format will be the same as in the existing program - a vector of length 4 each member in turn a vector of length 4. 0 will be used for the empty slot. For example, the board state should be represented like [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 0]] (This is a specific example of the goal board state) We do not wish to optimize or change this data structure. The UI should be carried over from the first project with minimal changes. It should be such that it asks the user to enter the start state in the above format, then it asks for the goal state (again in the same format). Then it asks the user to choose the algorithm to run: (1) Best First Search with heuristic 1 (2) Best First Search with heuristic 2 (3) Best First Search with heuristic 3 (4) halt. The program will loop through the options until option 4 is chosen. Please allow the user to enter any combination of {1, 2, 3} such as 123. Output format will be completely unchanged. The original code we will be working on is attached. If you need to ask a question about design intent or design choices, please ask before creating code.

RESPONSE:

I've reviewed your attached `main.cpp` and the specifications for the new project. We can preserve most of your existing implementation, including the `Board` data structure, `Node` structure, successor generation function, result reporting, and input format.

Before implementing anything, I have two design questions, primarily concerning Heuristic 2.

1. Reusing IDS for Heuristic 2

Your current implementation of IDS repeatedly calls DLS with increasing depth limits until it finds a particular goal state.

For Heuristic 2, however, we need the shortest distances from the mapped goal state g′g′ to all 102,960 possible Red/Green board states.

Calling your existing `IDS()` separately for every state would be extremely computationally expensive, particularly because the current DLS implementation does not maintain a visited-state collection.

My proposed approach:

Adapt the existing IDS/DLS implementation so that it can:

- Start at the mapped goal state g′g′.
- Explore the simplified state space using increasing depth limits.
- Track previously encountered states to avoid redundant exploration.
- Record each state's minimum distance from g′g′ in a lookup table.
- Stop once all 102,960 states have been recorded.

This retains the IDS approach instead of introducing a separate BFS algorithm for preprocessing.

Question 1: Are you comfortable modifying the existing IDS/DLS functionality to support this table-building process, provided we retain its underlying iterative-deepening search method?

2. Accounting for Heuristic 2 preprocessing time

You specified that the Heuristic 2 lookup table has a one-time construction cost.

I propose building this table when the user first selects algorithm 2 or 3 and retaining it for subsequent searches using the same goal state.

However, this introduces a question concerning your existing output:

```
The number of states removed: ... The maximum size of the queue
(stack): ... CPU time: ... seconds
```

There are two possibilities:

Option A: Include the time spent constructing the Heuristic 2 lookup table in the CPU time of the first search that requires it.

Option B: Exclude table construction from search CPU time, so the reported measurements represent only the actual A* search. The preprocessing would still occur, but its time would not be included in those measurements.

Question 2: Which option would you prefer?

Other implementation details

The remaining design choices appear sufficiently clear from your specifications.

I'll follow these conventions:

- Search algorithm: A* best-first search, with f(n)=g(n)+h(n)f(n)=g(n)+h(n), using the provided `Improve` logic. I'll treat the Open, Closed, and previously unseen cases as three mutually exclusive branches.
- Path cost: Each legal tile movement has a cost of 1, so the existing node depth can represent g(n)g(n).
- Heuristic 1: Manhattan distance, computed relative to the user-supplied goal.
- Heuristic 2: Exact distances in the simplified Red/Green puzzle.
- Heuristic 3: h3(n)=max(h1(n),h2(n))h3(n)=max(h1(n),h2(n)).
- Successor generation: Reuse `generateSuccessors()` and preserve U, D, L, R ordering.
- Board representation: Continue using `vector<vector<int>>` without changing the 4×4 board representation.
- User interface: Accept selections such as `1`, `2`, `3`, `12`, `13`, `23`, or `123`, execute the selected searches, and then redisplay the menu until the user selects `4`.
- Output: Preserve your existing `SearchResult` structure and the output format implemented by `printResult()`.

I also intend to preserve the current project scope without introducing solvability/parity validation.

Once we've settled those two questions, we can implement the project incrementally, beginning with the A* search algorithm and Heuristic 1, and test each component before proceeding to Heuristics 2 and 3.

PROMPT:
For question 1, we are comfortable modifying IDS to support the table building, as that table is needed in this design. For question 2, we will exclude table construction from search CPU time, the measurements will include only the A* search itself.

RESPONSE:

Both design decisions are settled. We'll proceed with the following approach.

1. Heuristic 2 preprocessing

We will modify the existing IDS/DLS implementation to construct the lookup table containing the shortest distances to all 102,960 simplified Red/Green puzzle states.

- Reuse the existing successor generation function.
- Preserve the iterative-deepening approach.
- Introduce visited-state tracking appropriate for constructing the table.
- Build the table only when Heuristic 2 or 3 is first requested.
- Reuse the completed table for subsequent searches with the same goal state.

The original 4×4 `vector<vector<int>>` board representation will remain unchanged.

2. CPU time measurements

Only A search execution will be timed.*

The lookup table will be constructed before starting the search timer. This ensures that measurements for Heuristics 1, 2, and 3 are directly comparable.

The existing output format will remain unchanged, including the number of states removed and maximum frontier size.

3. Implementation sequence

We'll implement and test the project incrementally:

1. A search and Heuristic 1 (Manhattan distance)* — implement and validate optimal paths, move sequences, and search statistics.
2. Heuristic 2 — adapt IDS/DLS for the Red/Green lookup table and verify the resulting distances.
3. Heuristic 3 — implement the maximum of Heuristics 1 and 2.
4. User interface — implement the repeating menu and combined selections such as `123`.
5. Final testing — verify correctness, optimality, and output consistency across all three heuristics.

We'll retain the existing code wherever practical and avoid introducing functionality beyond the project's requirements.

The next step is implementing A with Heuristic 1*, using your supplied `Improve` pseudocode and existing `Node`, `SearchResult`, and `generateSuccessors()` functionality. We'll test that implementation before adding the more complex Heuristic 2 preprocessing.

Step 1 Complete: A* Search with Heuristic 1

I've implemented A search using the Manhattan distance heuristic* in your existing C++ program.

Download updated main_astar_h1.cpp

Your original `main.cpp` has been preserved.

1. Changes implemented

- Added `manhattanDistance()` to calculate Heuristic 1 relative to the user-supplied goal.
- Added `AStar()` following your provided pseudocode.
- Added `Improve()` with separate handling for successors already in Open, in Closed, or previously undiscovered.
- Implemented Open as an ordered set, selecting nodes with the lowest $f(n) = g(n) + h(n)$.
- Preserved your existing `Node`, `SearchResult`, `Board`, and `generateSuccessors()` implementations.

The Manhattan distance calculation excludes the blank tile (`0`).

2. Temporary menu change

For this implementation stage, I've preserved the original menu and added:

5. A* (Heuristic 1: Manhattan distance)

This allows you to compare A* against your existing BFS implementation before we replace the menu with the final interface.

3. Testing results

The updated program compiled successfully using C++17, with no compiler warnings.

Test	Expected moves	Actual moves	Result
Start equals goal	0	0	Pass
One move from goal	1	1	Pass
Two moves from goal	2	2	Pass
Three moves from goal	3	3	Pass
Custom goal state	1	1	Pass

I also tested 12 additional puzzle configurations and compared A* against your existing BFS algorithm. All 12 returned identical optimal solution lengths.

4. Test this configuration yourself

Start state:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,0,14,15]]

Goal state:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]

Select option 5.

Expected solution:

The solution found.

The number of moves: 2

The sequence of moves: RR

The number of states removed: 3

The maximum size of the queue (stack): 4

CPU time will vary by machine.

5. Next step

I recommend testing the updated program in your own environment before proceeding.

Once you've confirmed the results, we'll implement Heuristic 2, beginning with the Red/Green board mapping and modifications needed to reuse IDS for lookup-table construction.

PROMPT:

I see the new code, in the future please create no comments in the code.

I notice this function

```
auto heuristic = [&goal](const Board& state) {
 return manhattanDistance(state, goal);
 };
```

is this a lambda function? Can you explain this code to us, we are not familiar with this style

RESPONSE:

Yes, that is a lambda function in C++. A lambda is essentially a function that can be defined directly inside another function without giving it a traditional function name.

Since you aren't familiar with this syntax, I recommend avoiding lambda functions in our implementation unless they're necessary.

Let's examine what this code does.

1. Understanding the lambda

Your code is:

```
auto heuristic = [&goal](const Board& state) {

    return manhattanDistance(state, goal);

};
```

We can break it into four parts.

Part 1: `auto heuristic`

The `auto` keyword tells C++ to determine the variable's type automatically.

Here, `heuristic` will refer to the lambda function we are creating.

Part 2: `[&goal]`

This is called the capture list.

It allows the lambda to access the `goal` variable defined in the surrounding function.

The `&` means that `goal` is captured by reference rather than copied.

Part 3: `(const Board& state)`

This is the lambda's parameter list, just like the parameters of a regular C++ function.

It accepts a `Board` named `state` by constant reference.

Part 4: The function body

```
{
    return manhattanDistance(state, goal);
}
```

This calls our Manhattan distance function using the current board and the desired goal board, then returns the calculated distance.

2. How the lambda is used

After defining the lambda, we can write:

```
int h = heuristic(current.state);
```

This effectively executes:

```
int h = manhattanDistance(current.state, goal);
```

Both produce the same result.

3. Do we need a lambda here?

No. In this case, the lambda is unnecessary.

We already have a function:

```
int manhattanDistance(const Board& state, const Board& goal);
```

We can call that function directly instead:

```
int h = manhattanDistance(current.state, goal);
```

This is simpler, easier to understand, and perfectly appropriate for our project.

The lambda was primarily a convenience that allowed us to call the heuristic using only one argument.

4. Our approach going forward

For this project, I suggest:

- Avoid lambda functions.
- Prefer ordinary, explicitly named C++ functions.
- Use syntax and programming techniques consistent with your existing BFS, DLS, and IDS implementations.
- Include no comments in newly written code, as requested.
- Preserve the algorithms and data structures we have already agreed upon.

We can replace the lambda with direct calls to `manhattanDistance()` without changing the search algorithm's behavior.

Before implementing Heuristic 2, I recommend making that small change to A* so the implementation remains consistent with the C++ techniques you're learning.

PROMPT:
make that small change, show the code here in the window without creating a new file

RESPONSE:
We can eliminate the lambda entirely and call `manhattanDistance()` directly. This requires small changes in four locations of your existing code.

We will not change the algorithm's behavior or create a new file.

1. Update the function declarations

Replace the existing `Improve()` and `AStar()` declarations with:

```
void Improve(const Node& u, const Node& v,
             const Board& goal, AStarData& data);


SearchResult AStar(const Board& start, const Board& goal);
```

We no longer need to pass a function as a parameter.

2. Update case 5 in `main()`

Replace the entire `case 5` block with:

```
case 5:
{
    SearchResult result = AStar(start, goal);
    printResult("A* (Manhattan distance)", result);
    break;
}
```

The lambda has been removed.

3. Update the `Improve()` function

Replace the existing function with:

```
void Improve(const Node& u, const Node& v,
```

```
        const Board& goal, AStarData& data) {


int tentativeG = u.depth + 1;
auto openEntry = data.openPositions.find(v.state);


if (openEntry != data.openPositions.end()) {
    if (tentativeG < data.bestG[v.state]) {
        data.open.erase(openEntry->second);
        data.openPositions.erase(openEntry);
        data.bestG[v.state] = tentativeG;
        insertIntoOpen(data, v,
                tentativeG + manhattanDistance(v.state, goal));
    }
}
else if (data.closed.count(v.state) != 0) {
    if (tentativeG < data.bestG[v.state]) {
        data.closed.erase(v.state);
        data.bestG[v.state] = tentativeG;
        insertIntoOpen(data, v,
                tentativeG + manhattanDistance(v.state, goal));
    }
}
else {
    data.bestG[v.state] = tentativeG;
```

```
        insertIntoOpen(data, v,
                tentativeG + manhattanDistance(v.state, goal));
    }
}
```

The important change is that every previous call to:

heuristic(v.state)

has been replaced with:

manhattanDistance(v.state, goal)

4. Update the `AStar()` function

Replace the existing function with:

```
SearchResult AStar(const Board& start, const Board& goal) {
    SearchResult result;
    clock_t startTime = clock();
    AStarData data;

    Node root = createRootNode(start);
    data.bestG[root.state] = 0;
    insertIntoOpen(data, root, manhattanDistance(root.state, goal));
    result.maxFrontierSize = data.open.size();

    while (!data.open.empty()) {
        auto first = data.open.begin();
        Node current = first->node;
```

```
    data.openPositions.erase(current.state);
    data.open.erase(first);
    data.closed.insert(current.state);

    ++result.statesRemoved;

    if (current.state == goal) {
        result.solutionFound = true;
        result.numberOfMoves = current.depth;
        result.moveSequence = current.moves;
        result.cpuTime = static_cast<double>(clock() - startTime) / CLOCKS_PER_SEC;
        return result;
    }

    vector<Node> successors = generateSuccessors(current);

    for (const Node& child : successors) {
        Improve(current, child, goal, data);
    }

    if (data.open.size() > result.maxFrontierSize) {
        result.maxFrontierSize = data.open.size();
    }
}
```

```
    result.cpuTime = static_cast<double>(clock() - startTime) / CLOCKS_PER_SEC;
    return result;
}
```

Notice that we now pass `goal` directly into `Improve()`:

Improve(current, child, goal, data);

This gives `Improve()` everything it needs to calculate Manhattan distance without using a lambda.

5. Remove the unnecessary header

At the beginning of your program, you may remove:

#include <functional>

It is no longer needed.

One consideration for Heuristics 2 and 3

Our original design used a lambda to allow A* to accept different heuristic functions.

With this change, A* is temporarily specific to Manhattan distance.

When we implement Heuristics 2 and 3, we can instead use an integer parameter:

SearchResult AStar(const Board& start, const Board& goal, int heuristicChoice);

Then an ordinary function can determine which heuristic to calculate using `if` or `switch` statements. This will let us reuse the same A* implementation for all three heuristics without introducing lambda functions.

Next, I recommend recompiling and repeating the two-move Manhattan distance test to verify that these changes preserve the results before we implement Heuristic 2.

Step 2: Implementing Heuristic 2

We'll implement the Red/Green heuristic using your existing IDS approach and `generateSuccessors()` function.

We will preserve the original `vector<vector<int>>` board representation and use ordinary named C++ functions, without lambdas or new code comments.

1. Representing the simplified board

For Heuristic 2, we map each tile to one of three values:

Original tile	Simplified value
`0` (empty slot)	`0`
`1` through `8` (Red)	`1`
`9` through `15` (Green)	`2`

For example, the standard goal:

{{1, 2, 3, 4},

{5, 6, 7, 8},

{9, 10, 11, 12},

{13, 14, 15, 0}}

Becomes:

{{1, 1, 1, 1},

{1, 1, 1, 1},

{2, 2, 2, 2},

{2, 2, 2, 0}}

The simplified board still uses the exact same `Board` type.

2. Verification of the IDS approach

I tested an adapted depth-limited search that records distances as IDS increases its depth limit.

All simplified states discovered

102,960

Table entries

48

Final depth limit

0

Distance mismatches

For the standard goal configuration, I independently compared all 102,960 distances against breadth-first search. They matched exactly.

The table construction took approximately 22 seconds in the test environment. This is a one-time preprocessing operation for the selected goal, and its CPU time will be excluded from the A* search statistics as agreed.

3. Add the necessary header and function declarations

Add this header near the beginning of your program:

#include <unordered_map>

An `unordered_map` stores a value associated with a key, much like the `map` already used in A*. It is useful here because we'll be performing many table lookups.

Add these declarations with your existing function prototypes:

Board colorBoard(const Board& board);

string colorBoardKey(const Board& board);

void DLSForTable(const Board& mappedGoal, int depthLimit,

unordered_map<string, int>& table);

```
void IDSForTable(const Board& goal,
                 unordered_map<string, int>& table);

int heuristic2Distance(const Board& state,
                       const unordered_map<string, int>& table);

int heuristicValue(const Board& state, const Board& goal,
                   int choice, const unordered_map<string, int>& table);
```

4. Add the Red/Green mapping and IDS functions

Place these functions after your existing `generateSuccessors()` implementation.

```
Board colorBoard(const Board& board) {
    Board colored = board;

    for (int row = 0; row < 4; ++row) {
        for (int col = 0; col < 4; ++col) {
            if (colored[row][col] >= 1 && colored[row][col] <= 8) {
                colored[row][col] = 1;
            }
            else if (colored[row][col] >= 9 && colored[row][col] <= 15) {
                colored[row][col] = 2;
            }
        }
    }
```

```
    return colored;
}

string colorBoardKey(const Board& board) {
    string key;

    for (int row = 0; row < 4; ++row) {
        for (int col = 0; col < 4; ++col) {
            key += char('0' + board[row][col]);
        }
    }

    return key;
}

void DLSForTable(const Board& mappedGoal, int depthLimit,
                 unordered_map<string, int>& table) {
    stack<Node> frontier;
    unordered_map<string, int> visitedDepth;

    frontier.push(createRootNode(mappedGoal));

    while (!frontier.empty()) {
```

```
Node current = frontier.top();
frontier.pop();

string key = colorBoardKey(current.state);
auto visited = visitedDepth.find(key);

if (visited != visitedDepth.end() &&
    visited->second <= current.depth) {
    continue;
}

visitedDepth[key] = current.depth;

if (table.find(key) == table.end()) {
    table[key] = current.depth;
}

if (current.depth >= depthLimit) {
    continue;
}

vector<Node> successors = generateSuccessors(current);

for (auto it = successors.rbegin();
```

```
            it != successors.rend(); ++it) {
            frontier.push(*it);
        }
    }
}

void IDSForTable(const Board& goal,
                 unordered_map<string, int>& table) {
    Board mappedGoal = colorBoard(goal);
    table.clear();

    int depthLimit = 0;

    while (table.size() < 102960) {
        DLSForTable(mappedGoal, depthLimit, table);
        ++depthLimit;
    }
}

int heuristic2Distance(const Board& state,
                       const unordered_map<string, int>& table) {
    return table.at(colorBoardKey(colorBoard(state)));
}
```

There are two important details about this implementation.

First, we reuse IDS rather than introduce a new search algorithm. `IDSForTable()` repeatedly calls a depth-limited search, increasing the depth limit by one after each iteration. The existing successor generator is used, and the stack still visits successors in U, D, L, R order.

Second, we track the smallest depth at which a state has been encountered during each DLS iteration. This is important because a state encountered along a longer path must not prevent that same state from being explored along a shorter path.

The `colorBoardKey()` function creates a 16-character string from a simplified board. This is used only to identify entries in the lookup table; it does not replace or change your `Board` data structure.

5. Connect Heuristic 2 to A*

We can keep using the A* algorithm we've implemented.

First, add:

```
int heuristicValue(const Board& state, const Board& goal,
                   int choice, const unordered_map<string, int>& table) {
    if (choice == 1) {
        return manhattanDistance(state, goal);
    }

    return heuristic2Distance(state, table);
}
```

This lets the same A* algorithm use either heuristic, depending on the selected option.

Replace your existing `Improve()` and `AStar()` declarations with:

```
void Improve(const Node& u, const Node& v, const Board& goal,
             int choice, const unordered_map<string, int>& table,
             AStarData& data);
```

```
SearchResult AStar(const Board& start, const Board& goal, int choice,
                   const unordered_map<string, int>& table);
```

Then replace the existing implementations of those two functions with the following:

```
void Improve(const Node& u, const Node& v, const Board& goal,
             int choice, const unordered_map<string, int>& table,
             AStarData& data) {
   int tentativeG = u.depth + 1;
   auto openEntry = data.openPositions.find(v.state);

   if (openEntry != data.openPositions.end()) {
      if (tentativeG < data.bestG[v.state]) {
         data.open.erase(openEntry->second);
         data.openPositions.erase(openEntry);
         data.bestG[v.state] = tentativeG;

         insertIntoOpen(data, v,
            tentativeG + heuristicValue(v.state, goal, choice, table));
      }
   }
   else if (data.closed.count(v.state) != 0) {
      if (tentativeG < data.bestG[v.state]) {
         data.closed.erase(v.state);
         data.bestG[v.state] = tentativeG;
```

```
            insertIntoOpen(data, v,
                tentativeG + heuristicValue(v.state, goal, choice, table));
        }
    }
    else {
        data.bestG[v.state] = tentativeG;

        insertIntoOpen(data, v,
            tentativeG + heuristicValue(v.state, goal, choice, table));
    }
}

SearchResult AStar(const Board& start, const Board& goal, int choice,
                   const unordered_map<string, int>& table) {
    SearchResult result;
    clock_t startTime = clock();
    AStarData data;

    Node root = createRootNode(start);
    data.bestG[root.state] = 0;

    insertIntoOpen(data, root,
        heuristicValue(root.state, goal, choice, table));
```

```
result.maxFrontierSize = data.open.size();

while (!data.open.empty()) {
    auto first = data.open.begin();
    Node current = first->node;

    data.openPositions.erase(current.state);
    data.open.erase(first);
    data.closed.insert(current.state);

    ++result.statesRemoved;

    if (current.state == goal) {
        result.solutionFound = true;
        result.numberOfMoves = current.depth;
        result.moveSequence = current.moves;

        result.cpuTime =
            static_cast<double>(clock() - startTime) / CLOCKS_PER_SEC;

        return result;
    }
```

```
        vector<Node> successors = generateSuccessors(current);

        for (const Node& child : successors) {
            Improve(current, child, goal, choice, table, data);
        }

        if (data.open.size() > result.maxFrontierSize) {
            result.maxFrontierSize = data.open.size();
        }
    }

    result.cpuTime =
        static_cast<double>(clock() - startTime) / CLOCKS_PER_SEC;

    return result;
}
```

Notice that `AStar()` now accepts an integer specifying which heuristic to use. We haven't introduced lambda functions or changed the Open/Closed implementation.

6. Temporary menu changes

For now, retain the existing menu and add option 6:

cout << "6. A* (Heuristic 2: Red/Green lookup table)\n";

Declare the lookup table in `main()`, before the `switch` statement:

unordered_map<string, int> heuristic2Table;

Replace the existing case 5 and add case 6:

```
case 5:
{
    SearchResult result = AStar(start, goal, 1, heuristic2Table);
    printResult("A* (Manhattan distance)", result);
    break;
}

case 6:
{
    IDSForTable(goal, heuristic2Table);

    SearchResult result = AStar(start, goal, 2, heuristic2Table);
    printResult("A* (Red/Green heuristic)", result);
    break;
}
```

Because `IDSForTable()` finishes before `AStar()` begins, its construction time is excluded from the search CPU time.

We'll implement the final repeating menu and reuse of the lookup table across selections in the later UI step.

7. Testing configurations

I compiled and tested the complete integration with these two configurations.

Test 1: Standard goal, two moves

Start:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,0,14,15]]

Goal:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]

Passed — 2 moves, sequence `RR`

3 states removed; maximum frontier size 4.

Test 2: Nonstandard goal, one move

Start:

[[1,0,3,4],[2,5,6,7],[8,9,10,11],[12,13,14,15]]

Goal:

[[0,1,3,4],[2,5,6,7],[8,9,10,11],[12,13,14,15]]

Passed — 1 move, sequence `L`

2 states removed; maximum frontier size 3.

The output still uses your original `printResult()` function, without any changes to its format.

I recommend applying and testing these changes before moving on to Heuristic 3. Once Heuristic 2 is confirmed in your environment, implementing $h_3(s) = \max\{h_1(s), h_2(s)\}$ will require only a small addition to `heuristicValue()`.

Step 3: Implementing Heuristic 3

We are now ready to implement Heuristic 3, defined as:

$$h_3(s) = \max\{h_1(s), h_2(s)\}$$

This heuristic calculates both the Manhattan distance and the Red/Green lookup-table distance, then returns whichever value is larger.

Since we've already implemented both heuristics, we only need to make three small changes to the program.

1. Update `heuristicValue()`

First, make sure the following header is included:

#include <algorithm>

This provides the `max()` function.

Replace the existing `heuristicValue()` implementation with:

```
int heuristicValue(const Board& state, const Board& goal,
                   int choice, const unordered_map<string, int>& table) {

    if (choice == 1) {
        return manhattanDistance(state, goal);
    }

    if (choice == 2) {
        return heuristic2Distance(state, table);
    }

    int h1 = manhattanDistance(state, goal);
    int h2 = heuristic2Distance(state, table);

    return max(h1, h2);
}
```

Here's how this works:

- When `choice == 1`, we calculate Manhattan distance.
- When `choice == 2`, we retrieve the Red/Green distance from the lookup table.
- When `choice == 3`, we calculate both and return the larger value.

For example, suppose a particular board has:

$$h_1(s) = 8, h_2(s) = 10$$

Then:

$$h_3(s) = \max(8, 10) = 10$$

This gives A* the more informative of the two estimates.

We do not need to modify `AStar()` or `Improve()`. They already call `heuristicValue()` using the selected heuristic.

2. Add Heuristic 3 to the temporary menu

In `main()`, add one more menu option:

cout << "7. A* (Heuristic 3: Maximum of H1 and H2)\n";

This keeps our existing testing menu intact while we finish implementing the search algorithms.

3. Add case 7 to the switch statement

Inside the existing `switch (choice)` statement, add:

```
case 7:
{
    IDSForTable(goal, heuristic2Table);

    SearchResult result = AStar(start, goal, 3, heuristic2Table);
    printResult("A* (Maximum of H1 and H2)", result);
    break;
}
```

Notice the order of operations:

1. `IDSForTable()` builds the Heuristic 2 lookup table.
2. `AStar()` begins executing with Heuristic 3.
3. The result is displayed using the existing `printResult()` function.

Since the lookup table is constructed before entering `AStar()`, its construction time is excluded from the CPU time measurement, as we agreed.

4. Testing Heuristic 3

For our initial test, use the following board.

Start state:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,0,14,15]]

Goal state:

[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]

Run option `7`.

The expected solution is:

The solution found.

The number of moves: 2

The sequence of moves: RR

For this configuration, both Manhattan distance and the Red/Green heuristic give a distance of 2. Therefore:

$$h_3(s) = \max(2, 2) = 2$$

The solution should match those produced by Heuristics 1 and 2.

We can also verify the calculation of Heuristic 3 independently of A* by adding a temporary test:

int h1 = manhattanDistance(start, goal);

int h2 = heuristic2Distance(start, heuristic2Table);

int h3 = heuristicValue(start, goal, 3, heuristic2Table);

cout << "H1: " << h1 << '\n';

cout << "H2: " << h2 << '\n';

cout << "H3: " << h3 << '\n';

Run this only after `IDSForTable()` has constructed the table.

Expected output for this test

2

H1

2

H2

2

H3

The key property we want to verify is:

$$\boxed{h_3(s)\geq h_1(s)\quad\text{and}\quad h_3(s)\geq h_2(s)}$$

This must hold for every board state.

5. Where we stand

Component	Status
A* search algorithm	Implemented
Heuristic 1 — Manhattan distance	Implemented

Heuristic 2 — Red/Green IDS lookup	Implemented
Heuristic 3 — Maximum of H1 and H2	Code provided
Final repeating menu	Next
Final integration testing	Pending

No changes to the `Node` structure, board representation, successor generation, or result output are necessary for Heuristic 3.

Once you've compiled and verified this addition, our next step is the final user interface: replacing the temporary testing menu with options 1–4, accepting combinations such as `123`, and allowing repeated selections without rebuilding the Heuristic 2 lookup table unnecessarily.

PROMPT:

I have a question. I was under the impression A* uses a priority queue as an underlying data structure. Does this code implement that? I don't think so. I think it creates one implicitly.

RESPONSE:

You're correct to question this. A typically uses a priority queue*, but our current implementation does not use the C++ `priority_queue` container.

Instead, it uses a `set` that behaves like a priority queue.

Let's examine the difference.

1. What our implementation currently uses

In our `AStarData` structure, we have:

```
struct AStarData {
    using OpenSet = set<AStarEntry, AStarEntryCompare>;
    OpenSet open;
    map<Board, OpenSet::iterator> openPositions;
    set<Board> closed;
    map<Board, int> bestG;
    long long nextInsertionOrder = 0;
};
```

The important declaration is:

```
using OpenSet = set<AStarEntry, AStarEntryCompare>;
```

This creates an alias for a `set` that uses a custom comparison function.

The comparison function orders nodes by their f(n)f(n) values:

```
struct AStarEntryCompare {
    bool operator()(const AStarEntry& a, const AStarEntry& b) const {
        if (a.f != b.f) {
            return a.f < b.f;
        }
        return a.insertionOrder < b.insertionOrder;
    }
};
```

Consequently, the node with the smallest f(n)f(n) is always at the beginning of the set.

When A* executes:

```
auto first = data.open.begin();
Node current = first->node;
```

It retrieves the node with the smallest f(n)f(n), exactly as your A* pseudocode requires.

2. How does this differ from a priority queue?

C++ provides an explicit priority queue:

```
priority_queue<Node, vector<Node>, Compare> open;
```

However, the two containers have different properties.

Property	`set` (current implementation)	`priority_queue`
Access highest-priority element	Yes	Yes
Automatically maintains priority ordering	Yes	Yes

Remove highest-priority element	Yes	Yes
Remove an arbitrary element	Yes	No
Update an existing element's priority directly	No	No
Typical implementation	Balanced search tree	Binary heap

The significant distinction concerns your `Improve()` function.

Your pseudocode requires us to update a node's f(n)f(n) value if we find a shorter path.

With our current `set`, we accomplish this by removing the old entry and inserting the improved entry.

A standard C++ `priority_queue` doesn't support removing an arbitrary entry. We would generally handle this by inserting an improved copy and ignoring outdated entries when they are eventually removed.

3. Does our implementation satisfy A*?

Yes.

A* requires that Open allow us to select the node with the minimum value of:

f(n)=g(n)+h(n)f(n)=g(n)+h(n)

It doesn't require that we specifically use the C++ `priority_queue` container.

Our ordered set satisfies this requirement.

However, it would be more accurate to describe our implementation as A using an ordered set as its priority-based frontier*, rather than A* using `std::priority_queue`.

4. What I recommend for our project

Since you're implementing A* as part of a course assignment, I would lean toward using an explicit `priority_queue` if that is the data structure your course associates with best-first search.

For example:

priority_queue<AStarEntry, vector<AStarEntry>, AStarEntryCompare> open;

We would need to adjust the comparison logic to make the smallest f(n)f(n) the highest priority, since C++ priority queues are max-heaps by default.

We would also need to modify `Improve()` slightly to handle improved paths correctly.

This would make the implementation more directly reflect the data structure you're learning.

My suggestion is to settle this before completing the final user interface. Would you prefer to retain the ordered `set`, which already satisfies the algorithm's requirements, or change to an explicit C++ `priority_queue`?

PROMPT:

we need a min ordered p queue right? is there a way to do that in C++?

RESPONSE:

Yes. A requires a min-priority queue*, meaning the node with the smallest f(n)f(n) value must be removed first.

C++ supports this directly using `std::priority_queue` with a custom comparator.

1. Default priority queue in C++

By default, C++ implements a max-priority queue:

priority_queue<int> pq;

For example, inserting `5`, `2`, and `8` gives:

```
8, 5, 2
```

The largest value is removed first.

2. Creating a min-priority queue

We can instead write:

priority_queue<int, vector<int>, greater<int>> pq;

Now the smallest value is removed first.

pq.push(5);

pq.push(2);

pq.push(8);

cout << pq.top();

Output:

2

3. Applying this to A*

Our A* priority queue needs to hold nodes rather than integers.

We can define an entry that contains the node and its f(n)f(n) value:

```
struct AStarEntry {
    Node node;
    int f;
};
```

Then define a comparator:

```
struct Compare {
    bool operator()(const AStarEntry& a, const AStarEntry& b) const {
        return a.f > b.f;
    }
};
```

Notice the `>` operator. This makes the entry with the smallest `f` value appear at the top.

We can now declare our min-priority queue:

```
priority_queue<AStarEntry, vector<AStarEntry>, Compare> open;
```

To remove the node with minimum f(n)f(n):

```
AStarEntry current = open.top();
open.pop();
```

This implements the operation specified by your pseudocode:

Remove u from Open with minimum f(u).

4. One important consideration: `Improve()`

There is one complication.

Your `Improve()` pseudocode requires updating a node's priority when a better path is discovered.

C++ `priority_queue` does not allow us to directly modify an existing entry's priority.

The usual solution is:

1. Insert a new entry with the improved f(n)f(n).
2. Keep track of the best known cost for each board state.
3. When an outdated entry is removed, skip it.

This is commonly called lazy deletion.

It preserves the behavior of A*, although the physical queue may temporarily contain multiple entries representing the same board state.

My recommendation: We should change our A* implementation to use an explicit min-priority queue.

It is a standard approach to implementing A*, and it aligns with the priority-queue data structure typically taught in algorithms courses.

Our next step would be to replace the `set`-based Open implementation with `std::priority_queue`, while preserving your `Improve()` pseudocode, the existing `Node` structure, and all three heuristics.

PROMPT:
if we were to change the implementation to a min priority queue would the efficiency change?

RESPONSE:

Yes. Changing from our current `set` implementation to a min-priority queue could improve efficiency, particularly when searching larger numbers of puzzle states.

However, there is a tradeoff involving how A* updates nodes when it discovers better paths.

1. Time complexity comparison

Both `std::set` and `std::priority_queue` are efficient at maintaining nodes in priority order.

Operation	Current `set`	Min-priority queue
Insert a node	O(logn)O(logn)	O(logn)O(logn)
Access minimum f(n)f(n)	O(1)O(1)	O(1)O(1)
Remove minimum	O(logn)O(logn)	O(logn)O(logn)
Remove a specific node	O(logn)O(logn)	Not directly supported
Update a node's priority	Remove and reinsert	Insert a new copy

Here, nn represents the number of entries in the container.

The asymptotic time complexity is essentially the same for ordinary insertions and removals.

The practical performance differences come from how these structures are implemented.

2. Why a priority queue could be faster

C++ typically implements `priority_queue` using a binary heap stored in a `vector`.

Our current `set` is typically implemented using a balanced binary search tree.

Min-priority queue (binary heap)

 Example heap ordered by f(n). The smallest value is at the root.

The heap has several practical advantages:

- It stores entries contiguously in memory.
- It generally requires fewer memory allocations.
- Its operations often have lower overhead than balanced-tree operations.

Consequently, a priority queue can be faster in practice even though both structures have similar big-O complexity.

3. The tradeoff: Updating nodes

Consider a state already in Open:

g(v)=10,h(v)=5g(v)=10,h(v)=5

Therefore:

f(v)=15f(v)=15

Suppose A* discovers a better path with:

g(v)=7,f(v)=12g(v)=7,f(v)=12

With our current `set`: We remove the entry with f=15f=15 and insert a new entry with f=12f=12. Only one entry represents that state in Open.

With `priority_queue`: We insert the improved entry with f=12f=12, but the old entry with f=15f=15 remains in the heap. When that outdated entry reaches the top, we detect it and discard it.

This means the priority queue may contain more entries than our current set.

As a result:

- Memory usage can increase due to duplicate entries.
- Some additional queue removals may occur.
- We need to distinguish outdated entries from valid ones.

Our implementation would also need to ensure that outdated removals do not incorrectly increase the reported number of states removed or the logical frontier size.

4. Which implementation would be more efficient for our 15-puzzle?

For our specific project:

Consideration	Likely advantage
Basic insertion/removal speed	`priority_queue`
Memory efficiency per entry	`priority_queue`
Avoiding duplicate entries	`set`
Updating an existing state	`set`
Simplicity of maintaining the exact Open set	`set`

The overall winner depends on how frequently A* discovers improved paths and how large Open becomes.

Also, changing the container would not affect Heuristic 2's IDS lookup-table construction time, because that preprocessing uses a separate search.

5. My recommendation

I recommend using an explicit min-priority queue for A*.

It aligns with the standard heap-based implementation of best-first search and is a natural fit for your course project.

We should retain the `bestG` and `closed` structures and adapt `Improve()` to use lazy deletion. The resulting algorithm will still find optimal solutions with our admissible heuristics.

We should not assume it is faster until we measure it.

The next useful step would be to replace the ordered set with `std::priority_queue` and compare the two implementations using identical puzzle configurations, checking both solution correctness and CPU time.

PROMPT:

we are going to proceed with the data structure as it exists, we will not change at this time.

RESPONSE:

We'll retain the existing `std::set`-based implementation of Open for A*.

This means:

- Open: Continue using `set<AStarEntry, AStarEntryCompare>` to maintain nodes in ascending order of f(n)f(n).
- Closed: Retain the existing `set<Board>`.
- Improve(): Continue removing and reinserting entries when a better path is discovered.
- A:* No changes to the search algorithm or its supporting data structures.
- Heuristics: Continue using the existing implementations of H1, H2, and H3.

The current implementation satisfies the priority-ordering requirements of A*, so there is no need to change it.

Our next step is completing the final user interface, including the repeating menu, combined selections such as 123, and reuse of the Heuristic 2 lookup table.

We'll continue making minimal changes to the existing program and keep new code free of comments.

Step 4: Implementing the Final User Interface

We will now replace the temporary testing menu with the final interface specified in your assignment.

The new interface will:

- Accept selections 1, 2, 3, or any combination such as 12, 23, 123, or 321.
- Execute the selected algorithms in the order entered.
- Redisplay the menu after the searches finish.
- Terminate only when the user enters 4.
- Construct the Heuristic 2 lookup table only once, when first needed.

- Preserve the original `printResult()` output format.

We do not need to modify the A* algorithm, `Improve()`, or any of the three heuristics.

1. Replace the existing `main()` function

Assuming you've incorporated the Heuristic 2 and 3 changes from our previous steps, replace your current `main()` with the following:

```
int main()
{
    cout << "=============================================\n";
    cout << "       Sliding Tile Puzzle Solver\n";
    cout << "=============================================\n\n";

    Board start = readBoard(
        "Enter the START state in the format:\n"
        "[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]\n"
        "Start state: "
    );

    Board goal = readBoard(
        "\nEnter the GOAL state in the same format:\n"
        "Goal state: "
    );

    cout << "\nStart state:\n";
    printBoard(start);

    cout << "\nGoal state:\n";
    printBoard(goal);

    unordered_map<string, int> heuristic2Table;
    bool tableBuilt = false;

    string selection;

    while (true) {
        cout << "\n=============================================\n";
        cout << "Choose an algorithm:\n";
        cout << "1. Best First Search with Heuristic 1\n";
        cout << "2. Best First Search with Heuristic 2\n";
```

```
cout << "3. Best First Search with Heuristic 3\n";
cout << "4. Halt\n";
cout << "=============================================\n";
cout << "Selection: ";

if (!(cin >> selection)) {
    return 0;
}

if (selection == "4") {
    break;
}

bool selected[4] = {false, false, false, false};
bool valid = true;

for (char digit : selection) {
    if (digit < '1' || digit > '3') {
        valid = false;
        break;
    }

    int index = digit - '0';

    if (selected[index]) {
        valid = false;
        break;
    }

    selected[index] = true;
}

if (!valid) {
    cout << "Invalid algorithm selection.\n";
    continue;
}

if ((selected[2] || selected[3]) && !tableBuilt) {
    IDSForTable(goal, heuristic2Table);
    tableBuilt = true;
}

cout << '\n';
```

```
        for (char digit : selection) {
            int choice = digit - '0';

            SearchResult result = AStar(start, goal, choice, heuristic2Table);

            if (choice == 1) {
                printResult("A* (Manhattan distance)", result);
            }
            else if (choice == 2) {
                printResult("A* (Red/Green heuristic)", result);
            }
            else {
                printResult("A* (Maximum of H1 and H2)", result);
            }

            cout << '\n';
        }
    }

    return 0;
}
```

2. Understanding the changes

There are three important parts of the new implementation.

A. Reading multiple selections

Previously, we used an integer:

int choice;

cin >> choice;

Now we use:

string selection;

cin >> selection;

This allows us to examine each digit individually.

For example, if the user enters `123`, the `for` loop processes `'1'`, `'2'`, and `'3'` in succession.

The expression:

```
int choice = digit - '0';
```

converts a character representing a digit into its integer value. For example, `'2' - '0'` produces the integer `2`.

B. Reusing the lookup table

These variables are declared outside the menu loop:

```
unordered_map<string, int> heuristic2Table;
bool tableBuilt = false;
```

The condition:

```
if ((selected[2] || selected[3]) && !tableBuilt) {
    IDSForTable(goal, heuristic2Table);
    tableBuilt = true;
}
```

ensures that the table is constructed only when Heuristic 2 or 3 is selected and only if it has not already been constructed.

Because the start and goal states remain fixed throughout the program, we can safely reuse this table.

C. Repeating the menu

The menu is enclosed within:

```
while (true)
```

When the user enters `4`, the following statement terminates the loop:

```
if (selection == "4") {
    break;
```

}

Entering an invalid selection prints an error and returns to the menu. Repeated digits such as `112` are treated as invalid rather than executing an algorithm twice.

3. Testing the final menu

Use this configuration for initial testing.

Start:

`[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,0,14,15]]`

Goal:

`[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]`

Menu test checklist

Selection `{test.selection}` — Run H1 only

2 moves, RR

Selection `{test.selection}` — Run H2, then H3

Both should return 2 moves, RR

Selection `{test.selection}` — Run H1, H2, then H3

All should return 2 moves, RR

Selection `{test.selection}` — Run H3, H2, then H1

Tests execution order

Selection `{test.selection}` — Invalid selection

Menu redisplayed

Selection `{test.selection}` — Program terminates

No searches executed

0 of 6 tests marked complete

4. Important verification

The updated interface ensures that table construction occurs before calling `AStar()`.

Therefore, the existing timer inside `AStar()`:

clock_t startTime = clock();

will continue to measure only the A* search itself.

Also, we have not changed the ordered `set` used for Open.

At this point, all requested functionality has been covered in the implementation steps. The remaining task is final integration testing: compiling the complete updated source and verifying that all three heuristics return optimal solutions while reporting their search statistics consistently.

PROMPT:

for testing, we need a start state that requires 30 moves to reach the goal

RESPONSE:

Here is a verified 30-move puzzle for testing all three A* heuristics.

Start state

```
[[5,1,6,3],[13,12,2,4],[9,11,0,15],[14,7,8,10]]
```

Goal state

```
[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]
```

Verified solution

OPTIMAL SOLUTION LENGTH 30 moves

Verified

One optimal move sequence is:

```
DLULUURRDLURRDLDDRUULDDLLURRDR
```

I verified this using iterative deepening A* with the admissible Manhattan distance heuristic. No solution was found at depths 26 or 28, and a solution was found at depth 30.
Manhattan-distance parity rules out odd solution lengths for this configuration.

Expected results

Algorithm	Expected moves
A* with Heuristic 1	30
A* with Heuristic 2	30
A* with Heuristic 3	30

All three algorithms should return an optimal solution of exactly 30 moves. The move sequences may differ because multiple optimal paths can exist.

Recommended test: Enter 123 in your final menu and compare the number of states removed, maximum frontier size, and CPU time for all three heuristics. This will give us a more meaningful performance comparison than our earlier two-move tests.

PROMPT:

Pasted markdown.mdDocument

let me provide you with the updated code. It is currently running 123 - it built the table, then ran test 1, but test 2 is taking forever.

RESPONSE:

I've reviewed your updated code, and I found an important reason why Heuristic 2 is taking considerably longer than Heuristic 1 on the 30-move puzzle.

The problem is most likely the strength of Heuristic 2, rather than the lookup table or the menu.

1. Comparing the heuristic values

For the 30-move start state we selected:

```
[[5,1,6,3],[13,12,2,4],[9,11,0,15],[14,7,8,10]]
```

I independently calculated the heuristic values using the standard goal.

H1 — MANHATTAN

26

H2 — RED/GREEN

16

H3 — MAXIMUM

26

Heuristic 1 estimates that at least 26 moves remain, while Heuristic 2 estimates only 16.

Both are admissible lower bounds, but the Red/Green heuristic provides substantially less information for this particular state.

That matters because A prioritizes nodes using f(n)=g(n)+h(n)f(n)=g(n)+h(n).* A weaker heuristic generally causes A* to investigate many more states before finding the goal.

Heuristic 2 can therefore be dramatically slower than Heuristic 1, even though its lookup-table calculations are efficient.

2. I reproduced the slowdown using your code

I reconstructed and compiled your uploaded C++ program and ran the A* implementation against the same 30-move puzzle.

Here are the results:

Search	States removed	Result
Heuristic 1	2,482	Solved in 30 moves
Heuristic 2	200,000	Still searching
Heuristic 3	2,237	Solved in 30 moves

I stopped the Heuristic 2 diagnostic run at 200,000 removed states. At that point, its Open set contained approximately 196,000 entries, and the process had used about 338 MB of memory.

This strongly suggests the search is not stuck in an infinite loop. It is exploring a much larger portion of the puzzle's state space.

Your `std::set` implementation is functioning as a minimum-ff frontier, so I do not recommend replacing it to address this issue.

3. A better 30-move test for all three heuristics

I generated another start state that is also exactly 30 moves from the standard goal, but is much more suitable for testing Heuristic 2.

New start state:

`[[15,14,10,4],[1,0,2,11],[5,9,3,7],[13,12,6,8]]`

Goal state:

`[[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]]`

For this configuration:

h1(s)=30,h2(s)=30,h3(s)=30h1(s)=30,h2(s)=30,h3(s)=30

I tested this state using your A* implementation with an independently constructed equivalent lookup table.

Heuristic	Moves	States removed	Maximum Open size
H1	30	117	129
H2	30	59	83
H3	30	36	51

All three searches completed successfully.

This configuration allows you to verify that all three algorithms function correctly on a 30-move puzzle without requiring an enormous search.

4. One minor issue in your code

I also noticed that your forward declaration for `Improve()` has its parameters in a different order from the actual function definition.

Your declaration should be:

```
void Improve(const Node& u, const Node& v, const Board& goal,
        int choice, const unordered_map<string, int>& table,

        AStarData& data);
```

This mismatch is not causing the slowdown, because your existing program compiles and uses the correct definition. Nevertheless, I recommend correcting it.

5. What I recommend

You can stop the currently running search if it is consuming excessive memory.

I would not modify A*, the lookup table, or the underlying data structures at this point. The evidence indicates that Heuristic 2's performance on the original puzzle is a consequence of its relatively weak distance estimate.

Next, run 123 using the new 30-move start state above. If your results match the 30-move solution lengths and the state counts shown here, we'll have a useful baseline before deciding whether additional performance testing is necessary.

PROMPT:

========== Sliding Tile Puzzle Solver ========== Enter the START state in the format: [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]] Start state: [[15,14,10,4],[1,0,2,11],[5,9,3,7],[13,12,6,8]] Enter the GOAL state in the same format: Goal state: [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,0]] Start state: [15 14 10 4] [1 0 2 11] [5 9 3 7] [13 12 6 8] Goal state: [1 2 3 4] [5 6 7 8] [9 10 11 12] [13 14 15 0] ========== Choose an algorithm: 1. Best First Search - h1(s) 2. Best First Search - h2(s) 3. Best First Search - h3(s) 4. Halt ========== Selection: 123 ========== Best First Search - Heuristic 1 (Manhattan distance) Results ========== The solution found. The number of moves: 30 The sequence of moves: ULDDRUURDDDLUUURDDLURRDDLUURDD The number of states removed: 117 The maximum size of the queue (stack): 129 CPU time: 0.011231 seconds ==========
========== Best First Search - Heuristic 2 (Red/Green heuristic) Results ========== The solution found. The number of moves: 30 The sequence of moves: ULDDRUURDDDLUUURDDLURRDDLUURDD The number of states removed: 59 The maximum size of the queue (stack): 83 CPU time: 0.003567 seconds ==========
========== Best First Search - Heuristic 3 (Maximum of H1 and H2) Results ========== The solution found. The number of moves: 30 The sequence of moves: ULDDRUURDDDLUUURDDLURRDDLUURDD The number of states removed: 36 The maximum size of the queue (stack): 51 CPU time: 0.001967 seconds ==========
========== Choose an algorithm: 1. Best First Search - h1(s) 2. Best First Search - h2(s) 3. Best First Search - h3(s) 4. Halt ========== Selection: Why would heuristic 3 beat heuristic 2? shouldnt it just be the same result, plus a tiny overhead for max()?

RESPONSE:

Your intuition makes sense, but there's an important distinction: Heuristic 3 does not simply perform Heuristic 2 and then call `max()`. It can actually cause A to explore fewer states.*

Your results demonstrate exactly why Heuristic 3 is useful.

1. What makes Heuristic 3 different?

Recall our definitions:

h1(s)=Manhattan distanceh1(s)=Manhattan distance

h2(s)=Red/Green lookup distanceh2(s)=Red/Green lookup distance

h3(s)=max{h1(s),h2(s)}h3(s)=max{h1(s),h2(s)}

The key observation is:

Heuristic 3 is always at least as large as either Heuristic 1 or Heuristic 2.

Consider an intermediate state where:

h1(s)=22,h2(s)=15h1(s)=22,h2(s)=15

Then:

h3(s)=max(22,15)=22h3(s)=max(22,15)=22

If we used Heuristic 2 alone, A* would underestimate the remaining cost by a larger amount.

Heuristic 3 uses the more informative estimate.

2. Why does that reduce the number of states explored?

Remember that A* selects the node with the smallest:

f(n)=g(n)+h(n)f(n)=g(n)+h(n)

Suppose A* encounters a state with:

g(n)=10g(n)=10

And its heuristic values are:

h1(n)=22,h2(n)=15h1(n)=22,h2(n)=15

Compare the two algorithms:

Calculation	Heuristic 2	Heuristic 3
g(n)g(n)	10	10
h(n)h(n)	15	22
f(n)f(n)	25	32

We know the optimal solution is 30 moves.

Under Heuristic 2, this node has f=25f=25, making it attractive to A*.

Under Heuristic 3, the same node has f=32f=32. A* can find the optimal 30-move solution without ever expanding this node.

This is the important difference: Heuristic 3 can prevent A from spending time exploring states that Heuristic 2 incorrectly makes look promising.*

3. Your actual results demonstrate this

States removed from Open

03570105140

H1

H2

H3

- H1: States removed 117
- H2: States removed 59
- H3: States removed 36

Fewer states removed generally means less search work.

Comparing Heuristic 3 with Heuristic 2:

FEWER STATES REMOVED

39%

36 instead of 59

LOWER MEASURED CPU TIME

45%

0.001967 vs. 0.003567 seconds

Although Heuristic 3 performs additional calculations for each heuristic evaluation, it examines substantially fewer states.

That savings can outweigh the cost of calculating Manhattan distance and calling `max()`.

The CPU measurements are very short, so the precise time difference may fluctuate between runs. The difference in states removed is more informative.

4. But aren't H1, H2, and H3 all equal to 30 for our starting state?

Yes! And this is an excellent observation.

For your starting board:

h1(s)=h2(s)=h3(s)=30h1(s)=h2(s)=h3(s)=30

However, A doesn't evaluate the heuristic only for the starting state.*

It calculates heuristic values for newly discovered states throughout the search.

As the algorithm explores different branches, Manhattan distance and the Red/Green distance can produce different estimates.

For states along an optimal path in this particular puzzle, both estimates remain exact. But on alternative branches, they may differ considerably.

It is those alternative branches where Heuristic 3 gains its advantage.

5. Does Heuristic 3 always outperform Heuristic 2?

Not necessarily.

We can establish that:

$h3(s) \geq h2(s) h3(s) \geq h2(s)$

for every state.

Therefore, Heuristic 3 is at least as informative as Heuristic 2 in terms of its estimated remaining distance.

However, this does not guarantee lower CPU time in every experiment because:

- Heuristic 3 performs more calculations.
- Both heuristics may return identical values for many states.
- Tie-breaking can influence the number of states expanded.
- Small variations in execution time can affect measurements.

Both are admissible heuristics, so A* can still produce optimal solutions.

The important conclusion

Your implementation is demonstrating the reason we use:

$h3(s) = \max\{h1(s), h2(s)\} h3(s) = \max\{h1(s), h2(s)\}$

We aren't combining the two heuristics merely for accuracy at the starting state. We're giving A* a stronger estimate throughout the entire search space.

Your results are consistent with the expected behavior of all three heuristics.

A useful next experiment would be to print the values of H1 and H2 for a few intermediate states where they disagree. That would let us observe directly how Heuristic 3 changes A*'s decision-making.